

Day 1 — Models & Project Setup

Django E-Commerce Project | Part 1: Build Steps with Syntax | Part 2: Explanations, Checks & Reflection

0. Today's Goal

By the end of today you should have a running Django project with a correctly designed database schema for the whole store — **Category, Product, Order, OrderItem** — migrated into SQLite, visible in the admin, and queryable from the Django shell. No views or templates yet; that is Day 2. Today is about getting the *data shape* right, because every later day sits on it.

Block	What you do	Time
A	Environment: venv, Django, python-decouple, Pillow, Git	30 min
B	Project + apps (config, store, orders), settings, .env	45 min
C	Schema design on paper (before any code)	30 min
D	Write models: Category, Product	60 min
E	Write models: Order, OrderItem	60 min
F	Migrations + basic admin registration	45 min
G	Shell practice: QuerySets, related_name, lookups	45 min
H	Blank-Page Test + LEARNING.md + commit	30 min

How to use this guide (struggle-first): for each step, read the **Your task** line and the syntax pattern, then write the code yourself. Only open the **Reference** block after you have a working attempt (or have been stuck for ~15 minutes). Bring errors and your code to me for review — I'll explain, not just fix.

Gateway assumption: Day 1 models are gateway-neutral. The Order model has a generic payment_ref field that will hold Khalti's pidx or eSewa's transaction_uuid on Day 5. Khalti v2 is still the recommended sandbox — confirm before Day 5.

PART 1 — Build Steps

Step A — Environment

Your task: create a project folder, a virtual environment, install packages, and freeze them.

```
terminal
mkdir django-ecom && cd django-ecom
python -m venv venv

# activate
venv\Scripts\activate      # Windows (PowerShell / cmd)
source venv/bin/activate   # macOS / Linux

pip install django python-decouple pillow
pip freeze > requirements.txt
python -m django --version  # sanity check
```

Create `.gitignore` **before** your first commit so secrets and the DB never reach GitHub:

```
.gitignore
venv/
__pycache__/*
*.pyc
.env
db.sqlite3
media/
```

Step B — Project, Apps, Settings

Your task: create the project in the current folder (note the trailing dot) and two apps.

```
terminal
django-admin startproject config .      # '.' = no extra nested folder
python manage.py startapp store         # Category, Product (catalog)
python manage.py startapp orders        # Order, OrderItem (checkout/payment)
python manage.py runserver              # visit http://127.0.0.1:8000 -> rocket page
```

Why two apps? A Django app is a cohesive feature unit. The catalog and the order/payment lifecycle change for different reasons, so they live apart. The session cart (Day 3) will get its own cart app.

Expected layout:

```
django-ecom/
├── config/          settings.py, urls.py, wsgi.py, asgi.py
├── store/           models.py, admin.py, apps.py, migrations/
├── orders/          models.py, admin.py, apps.py, migrations/
├── manage.py
├── requirements.txt
├── .env             (gitignored)
└── .gitignore
```

B.1 Secrets via `.env` (python-decouple)

Generate a fresh secret key and put it in `.env` (same folder as `manage.py`):

```
terminal
python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"
```

`.env`

```
SECRET_KEY=paste-the-generated-key-here
DEBUG=True
ALLOWED_HOSTS=127.0.0.1,localhost
```

Syntax pattern — `config(NAME, default=..., cast=...):`

`config/settings.py` (edit these parts)

```
from pathlib import Path
from decouple import config, Csv

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = config("SECRET_KEY") # no default -> crash if missing (good)
DEBUG = config("DEBUG", default=False, cast=bool) # "True" string -> True bool
ALLOWED_HOSTS = config("ALLOWED_HOSTS", default="127.0.0.1,localhost", cast=Csv())

INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    # local apps
    "store",
    "orders",
]

TIME_ZONE = "Asia/Kathmandu"
USE_TZ = True

STATIC_URL = "static/"
MEDIA_URL = "/media/" # URL prefix for uploaded files
MEDIA_ROOT = BASE_DIR / "media" # folder on disk for uploaded files
```

Serve uploaded product images in development only:

`config/urls.py`

```
from django.conf import settings
from django.conf.urls.static import static
from django.contrib import admin
from django.urls import path

urlpatterns = [
    path("admin/", admin.site.urls),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Step C — Design the Schema on Paper First

Your task: before writing code, draw the four tables and answer the design questions below. Write your answers in `LEARNING.md`.

Category 1	—	*	Product 1	—	*	OrderItem	*	—	1	Order	*	—	1	User
(name, slug)			(name, slug,			(price snapshot,				(status, total,				(Django built-in)
			price, stock,			quantity)				payment_ref,				
			image, active)							shipping info)				

- Why does `OrderItem` store its own price when `Product` already has one?
- If an admin deletes a `Category` that still has `Products`, what *should* happen? What about deleting a `Product` that has been ordered?

- Why is money a `DecimalField` and never a `FloatField`?
- Which field will tell you whether an order is safe to ship? Who is allowed to change it (hint: Day 5)?
- Why is there no Cart model? (Hint: Day 3 uses `request.session`.)

Step D — Catalog Models (`store/models.py`)

Your task: write `Category` and `Product`. Syntax patterns you need:

syntax

```
class ModelName(models.Model):
    field_name = models.FieldType(option=value, ...)

    class Meta:
        # model-level options (ordering, names, indexes)
        ordering = ["field"]

    def __str__(self):
        # how the object prints in admin / shell
        return self.field_name

# Relationship pattern (the "many" side holds the ForeignKey):
parent = models.ForeignKey(ParentModel, on_delete=models.PROTECT, related_name="children")
```

Field type	Use for	Key options
<code>CharField</code>	short text (name)	<code>max_length</code> (required), <code>unique</code>
<code>SlugField</code>	URL-safe id (<i>samsung-a15</i>)	<code>unique=True</code> ; used in Day 2 URLs
<code>TextField</code>	long text	<code>blank=True</code> = optional in forms
<code>DecimalField</code>	money	<code>max_digits</code> , <code>decimal_places</code>
<code>PositiveIntegerField</code>	stock, quantity	<code>default=0</code>
<code>ImageField</code>	product photo	<code>upload_to=</code> ; needs Pillow
<code>BooleanField</code>	flags (<code>is_active</code>)	<code>default=True</code>
<code>DateTimeField</code>	timestamps	<code>auto_now_add</code> (on create), <code>auto_now</code> (every save)

Reference – store/models.py

```
from django.db import models

class Category(models.Model):
    name = models.CharField(max_length=100, unique=True)
    slug = models.SlugField(max_length=110, unique=True)

    class Meta:
        ordering = ["name"]
        verbose_name_plural = "categories" # admin shows "categories", not "categorys"

    def __str__(self):
        return self.name

class Product(models.Model):
    category = models.ForeignKey(
        Category,
        on_delete=models.PROTECT, # can't delete a category that still has products
        related_name="products", # category.products.all()
    )
    name = models.CharField(max_length=200)
    slug = models.SlugField(max_length=220, unique=True)
    description = models.TextField(blank=True)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)
    image = models.ImageField(upload_to="products/", blank=True)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ["-created_at"] # '-' = descending (newest first)

    def __str__(self):
        return self.name

    @property
    def in_stock(self):
        return self.stock > 0
```

Step E — Order Models (orders/models.py)

Your task: write Order (with a status enum) and OrderItem. New syntax:

syntax

```
# Enum-style choices (Django 3.0+)
class Status(models.TextChoices):
    PENDING = "pending", "Pending" # CONSTANT = "stored value", "human label"

status = models.CharField(max_length=20, choices=Status.choices, default=Status.PENDING)

# Never import User directly in models -- reference the setting:
from django.conf import settings
user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=..., related_name="orders")

# Cross-app ForeignKey via "app_label.ModelName" string (avoids circular imports)
product = models.ForeignKey("store.Product", on_delete=models.PROTECT)
```

Reference – orders/models.py

```
from decimal import Decimal

from django.conf import settings
from django.core.validators import MinValueValidator
from django.db import models

class Order(models.Model):
    class Status(models.TextChoices):
        PENDING = "pending", "Pending payment"
        PAID = "paid", "Paid"
        FAILED = "failed", "Payment failed"
        CANCELLED = "cancelled", "Cancelled"

    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.PROTECT,          # keep financial records
        related_name="orders",             # request.user.orders.all() (Day 6 history page)
    )
    full_name = models.CharField(max_length=150)
    phone = models.CharField(max_length=20)
    address = models.CharField(max_length=255)

    total_amount = models.DecimalField(max_digits=10, decimal_places=2, default=Decimal("0.00"))
    status = models.CharField(max_length=20, choices=Status.choices, default=Status.PENDING)

    # Set on Day 5 ONLY after server-side verification with the gateway
    # (Khalti pidx / eSewa transaction_uuid)
    payment_ref = models.CharField(max_length=100, blank=True)
    paid_at = models.DateTimeField(null=True, blank=True)

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ["-created_at"]

    def __str__(self):
        return f"Order #{self.pk} ({self.get_status_display()})"

    def calculate_total(self):
        return sum((item.subtotal for item in self.items.all()), Decimal("0.00"))

class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE, related_name="items")
    product = models.ForeignKey(
        "store.Product", on_delete=models.PROTECT, related_name="order_items"
    )
    price = models.DecimalField(max_digits=10, decimal_places=2) # snapshot at purchase
    quantity = models.PositiveIntegerField(
        default=1, validators=[MinValueValidator(1)]
    )

    def __str__(self):
        return f"{self.quantity} x {self.product}"

    @property
    def subtotal(self):
        return self.price * self.quantity
```

Step F — Migrations + Admin

terminal

```
python manage.py makemigrations      # reads models.py -> writes migration files
python manage.py showmigrations      # [X] applied, [ ] pending
python manage.py sqlmigrate store 0001 # print the actual SQL
python manage.py migrate              # applies migrations to db.sqlite3
python manage.py createsuperuser
```

Your task: register all four models so you can add sample data. Day 6 does full customization — today, just enough to work with.

Reference – store/admin.py

```
from django.contrib import admin
from .models import Category, Product

@admin.register(Category)                # decorator form of admin.site.register(...)
class CategoryAdmin(admin.ModelAdmin):
    list_display = ["name", "slug"]
    prepopulated_fields = {"slug": ("name",)} # auto-fills slug while typing name

@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
    list_display = ["name", "category", "price", "stock", "is_active"]
    list_filter = ["category", "is_active"]
    prepopulated_fields = {"slug": ("name",)}
```

Reference – orders/admin.py

```
from django.contrib import admin
from .models import Order, OrderItem

class OrderItemInline(admin.TabularInline):    # edit items inside the order page
    model = OrderItem
    extra = 0

@admin.register(Order)
class OrderAdmin(admin.ModelAdmin):
    list_display = ["id", "user", "status", "total_amount", "created_at"]
    list_filter = ["status"]
    inlines = [OrderItemInline]
```

Run the server, open `/admin/`, and add **3 categories** and **at least 8 products** (you need this data for Day 2). Then try deleting a category that has products — you should get a PROTECT error. That's your schema working.

Step G — Shell Practice (QuerySets)

Your task: open `python manage.py shell` and run each line, predicting the result *before* pressing Enter.

python manage.py shell

```
from decimal import Decimal
from store.models import Category, Product

# CREATE
phones = Category.objects.create(name="Phones", slug="phones")
Product.objects.create(category=phones, name="Redmi 13", slug="redmi-13",
                        price=Decimal("21999.00"), stock=5)

# READ
Product.objects.all()                # QuerySet (lazy -- no SQL until used)
Product.objects.get(slug="redmi-13") # exactly one, else raises
Product.objects.filter(price__lt=30000) # field__lookup (lt, gt, lte, gte)
Product.objects.filter(name__icontains="redmi") # case-insensitive contains
Product.objects.filter(category__slug="phones") # follow the FK with double underscore
Product.objects.exclude(stock=0).order_by("price")

# REVERSE relation via related_name
phones.products.all()
phones.products.count()

# Avoid N+1 queries: JOIN the category in one SQL query
for p in Product.objects.select_related("category"):
    print(p.name, p.category.name)

# See the SQL Django generates
print(Product.objects.filter(price__lt=30000).query)

# UPDATE / DELETE
Product.objects.filter(slug="redmi-13").update(stock=10)
phones.delete() # -> ProtectedError. Expected!
```

Stretch: create an Order for your superuser with two OrderItems in the shell, then call `order.calculate_total()` and `order.get_status_display()`.

stretch hint

```
from django.contrib.auth import get_user_model
from orders.models import Order, OrderItem
User = get_user_model()
u = User.objects.first()
o = Order.objects.create(user=u, full_name="Test", phone="98XXXXXXX", address="Kathmandu")
p = Product.objects.first()
OrderItem.objects.create(order=o, product=p, price=p.price, quantity=2)
o.calculate_total(), o.get_status_display(), u.orders.count()
```

Step H — Commit

terminal

```
git init
git add .
git status          # .env, db.sqlite3, venv/ must NOT be listed
git commit -m "Day 1: setup, catalog + order models, admin"
git branch -M main
git remote add origin https://github.com/<you>/django-ecom.git
git push -u origin main
```

PART 2 — Explanations

1. MVT vs MVC — Where Today Fits

Django (MVT)	Role	ASP.NET Core MVC
Model (models.py)	Data + business rules, maps to tables	Model / Entity + DbContext
View (views.py)	Receives request, gets data, picks template	Controller
Template (.html)	Presentation	View (.cshtml)
URLconf (urls.py)	Routes URL -> view function	Routing / attribute routes

The naming trap: Django's **View** is ASP.NET's **Controller**. Django's "controller" is arguably the framework itself plus the URLconf. Today you only touch the **M**.

2. Project vs App

The **project** (config/) is configuration: settings, root URLs, WSGI/ASGI entry points. An **app** is a reusable feature package with its own models, admin, migrations, and (later) views and templates. An app does nothing until it's listed in `INSTALLED_APPS` — that is how Django discovers its models and migrations. Forgetting this gives the classic `No changes detected from makemigrations`.

3. Fields: null vs blank

```
null=True      # DATABASE level: column may store NULL
blank=True     # VALIDATION level: forms/admin allow the field to be empty

description = models.TextField(blank=True)          # text: use blank only -> stores ""
paid_at = models.DateTimeField(null=True, blank=True) # non-text: needs both
```

Convention: for string fields avoid `null=True`, otherwise "no value" has two representations (NULL and "").

4. ForeignKey, on_delete, related_name

A `ForeignKey` lives on the "many" side. Django creates a `category_id` column and gives you two directions:

```
product.category          # forward: Product -> its Category (one object)
category.products.all()   # reverse: Category -> its Products (via related_name)
# without related_name the reverse would be: category.product_set.all()
```

on_delete	Behaviour	Used here for
CASCADE	Delete children with parent	OrderItem when its Order is deleted
PROTECT	Refuse delete while children exist	Category->Product, Product->OrderItem, User->Order
SET_NULL	Set FK to NULL (needs <code>null=True</code>)	Not used today
RESTRICT	Like PROTECT, but allows cascades from elsewhere	Not used today

Why PROTECT on orders: orders are financial records. If deleting a product silently deleted past order lines, your sales history and payment audit trail would be corrupted. To retire a product, set

`is_active=False` instead.

5. Price Snapshot — Why `OrderItem.price` Exists

Prices change. If an order on Oct 5 showed `product.price` directly, raising the price on Oct 10 would rewrite what the customer paid. Copying the price at checkout makes an order **immutable history**. This also matters on Day 5: the amount you send to the gateway, the amount it confirms, and `order.total_amount` must all match — and that total must be computed on the server from DB prices, never from a value the browser sends.

6. Why `DecimalField`, Not `FloatField`

```
>>> 0.1 + 0.2
0.30000000000000004          # float: binary approximation
>>> from decimal import Decimal
>>> Decimal("0.1") + Decimal("0.2")
Decimal('0.3')                # exact
```

Always build Decimals from strings (`Decimal("0.1")`), not floats (`Decimal(0.1)` keeps the float error). Heads-up for Day 5: Khalti expects amounts in **paisa** (integer, Rs 1 = 100 paisa), so you'll convert with `int(total * 100)`.

7. TextChoices — Status as a State Machine

`Order.Status` stores a short string in the DB but gives you constants in Python (`Order.Status.PAID`) so typos become `AttributeErrors` instead of silent bugs. Django auto-adds `get_status_display()` for the human label. Think of the allowed transitions now:

```
PENDING --(server verifies payment with gateway)--> PAID
PENDING --(gateway reports failure / user cancels)--> FAILED / CANCELLED
PAID     --X--> anything set by the browser          # the Day 5 security rule
```

8. Migrations — What Actually Happens

- `makemigrations` compares your models to the last migration state and writes a Python file in `app/migrations/`. It does **not** touch the DB.
- `migrate` runs unapplied migration files against the DB and records them in the `django_migrations` table.
- Migration files **are committed** to Git — they are the schema's version history. The DB file is not.
- Adding a non-nullable field to a table with rows triggers a prompt: give a one-off default, or add `default=nullable=True` in the model.

9. QuerySets Are Lazy

`Product.objects.filter(...)` builds a query but runs no SQL. SQL runs when you iterate, slice with a step, call `len()`, `list()`, `.get()`, `.count()`, or print in the shell. This lets you chain filters cheaply. The **N+1 problem**: looping over products and touching `p.category` fires one extra query per product; `select_related("category")` turns that into a single JOIN. Same idea as EF Core's `.Include()`.

10. Secrets & settings.AUTH_USER_MODEL

SECRET_KEY signs sessions, CSRF tokens and password-reset links. If it leaks, attackers can forge signed data. On Day 5 your Khalti/eSewa secret key goes into the same .env. Referencing settings.AUTH_USER_MODEL instead of importing User keeps your models working if you ever swap in a custom user model.

11. Common Day 1 Errors

Error	Cause	Fix
UndefinedValueError: SECRET_KEY not found	.env missing or in wrong folder	Put .env next to manage.py
No changes detected	App not in INSTALLED_APPS	Add 'store', 'orders'
Cannot use ImageField because Pillow is not installed	Missing dependency	pip install pillow
no such table: store_product	Migrations not applied	makemigrations then migrate
fields.E120 max_length	CharField without max_length	Add max_length
ProtectedError	Deleting a parent with PROTECT children	Expected — deactivate instead
Reverse accessor clashes	Two FKs to the same model without related_name	Give each a unique related_name

12. Django vs ASP.NET Core MVC (Day 1 Edition)

Task	Django	ASP.NET Core
New project	django-admin startproject	dotnet new mvc
Define schema	models.py classes	Entity classes + DbContext + DbSet
Create migration	makemigrations	dotnet ef migrations add
Apply migration	migrate	dotnet ef database update
Config	settings.py (Python code)	appsettings.json + Program.cs
Secrets	.env + decouple	User Secrets / env vars
Navigation props	related_name (auto reverse)	Explicit collection properties
Eager loading	select_related / prefetch_related	.Include()
Admin UI	Built-in, free	None (scaffold CRUD pages)

13. Blank-Page Test (do this with the code closed)

- Write Product from memory: FK with on_delete + related_name, DecimalField with correct options, the two timestamp fields.
- Write Order.Status as TextChoices and the status field that uses it.
- Write the decouple lines for SECRET_KEY, DEBUG and ALLOWED_HOSTS.
- Write a shell query: all active, in-stock products in category slug *phones*, cheapest first, with categories JOINed.

- Explain aloud: *why price snapshot, why PROTECT, why Decimal, makemigrations vs migrate.*

If you can't do 4 of 5 without peeking, slow down before Day 2.

LEARNING.md entry

Day 1 — Models & Project Setup

- Built: ...
- Hardest part / what broke and why: ...
- Django-specific (ORM, settings, admin): ...
- Transferable (schema design, migrations, secrets, Decimal for money): ...
- Surprised me vs ASP.NET Core MVC: ...
- Blank-Page Test result: x/5

Done when: server runs • 4 models migrated • 3 categories + 8 products in admin • shell queries work • pushed to GitHub with .env excluded • LEARNING.md updated. **Next — Day 2:** catalog views, URLconf with slug converters, template inheritance.